

FORGES

Formation · Organisation · Gestion · Enrôlement · Suivi
Conception Technique & Guide de Développement

Version 1.8 · Avril 2026



Taux de complétude : 98%



Basé sur Specs v4.8



15 modules



148 règles métier



Table des Matières

1. Introduction
2. Standards de développement
3. Architecture en 5 couches
4. Ordre d'implémentation
5. Modules détaillés (MOD-01 à MOD-15)
6. Modèle de données (Prisma v3)
7. DTOs complets
8. Configuration centralisée
9. Schedulers
10. Machines à états
11. Bot Conseiller
12. Endpoints API
13. Swagger/OpenAPI
14. Stratégie de tests
15. Objectifs de couverture
16. Optimisation des appels
17. Setup et configuration
18. Gestion des erreurs
19. Glossaire technique
20. Générateur de tests unitaires

1. Introduction

1.1 Vision et Objectifs

FORGES est une plateforme web d'agrégation de formations certifiantes internationales, conçue pour le marché africain. Elle gère le cycle de vie complet d'une formation : création, planification, inscription, traitement des dossiers, paiement et suivi.



Acronyme FORGES

Formation · Organisation · **R**Gestion · **G**estion · **E**nrôlement · **S**uivi

1.2 Stack Technique

Couche	Technologie	Rôle
Backend	Node.js + Express + Prisma ORM	API REST, logique métier
Frontend	React + Vite + Tailwind CSS	Interface responsive
Base de données	PostgreSQL 16	Persistence principale
Cache	Redis 7	Sessions, rate limiting, files
Scheduler	node-cron + config centralisée	Tâches automatiques
Authentification	JWT + bcrypt (coût 12)	Access 1h, refresh 7j

2. Standards de Développement

2.1 Règles de nommage

Type	Règle	Exemple
Fichiers	kebab-case	<code>inscription-service.ts</code>
Classes	PascalCase	<code>InscriptionService</code>
Méthodes	camelCase, verbe+nom	<code>calculerCommission()</code>
Constantes	SCREAMING_SNAKE_CASE	<code>MAX_TENTATIVES</code>
DTOs	suffixe <code>Dto</code>	<code>InscriptionSessionDto</code>
Services	suffixe <code>Service</code>	<code>PaieementService</code>
Repositories	suffixe <code>Repository</code>	<code>DossierRepository</code>

DRY

Une fonction = une responsabilité. Pas de duplication > 3 lignes.

Single Responsibility

Une classe = un domaine métier.

Encapsulation

Propriétés privées = `private` / `#`.

Optimisation DB

Pas de N+1. Utiliser `include` ou `select`.

 **Règles bloquantes** : Toute fonction > 50 lignes justifiée | Pas de `any` | Appels BDD uniquement dans repositories

3. Architecture en 5 couches

COUCHE 1 - PRÉSENTATION
React Components · Pages · Widgets · i18n · Bot flottant
COUCHE 2 - API GATEWAY
Controllers · Middleware (JWT, RBAC, Rate Limit) · Validation DTO
COUCHE 3 - SERVICES MÉTIER
15 modules · 148 règles métier · Orchestration · Transactions
COUCHE 4 - ACCÈS DONNÉES
Repositories · Prisma ORM · Raw queries · Migrations
COUCHE 5 - SERVICES TRANSVERSAUX
Audit (MT-01) · Encryption (MT-02) · Email · Traduction · Scheduler

Règles de circulation

-  Controller → Service → Repository → Prisma
-  Controller → Repository (interdit)
-  Service → Prisma direct (passer par Repository)
-  Service → Service d'un autre module (via injection)

4. Ordre d'implémentation

 **Ordre de génération du code et des tests :**

Ordre d'implémentation du code

1. **MOD-11** (Transversal) – socle obligatoire
2. **MOD-01** (Authentification)
3. **MOD-02** (Comptes)
4. **MOD-03** (Formations)
5. **MOD-04** (Sessions)
6. **MOD-07** (Vouchers)
7. **MOD-05** (Inscriptions) – cœur fonctionnel
8. **MOD-06** (Paielements)
9. **MOD-09** (Espace Apprenant)
10. **MOD-10** (Espace Organisation)
11. **MOD-08** (Tableaux de bord)
12. **MOD-15** (Abonnements)
13. **MOD-12** (Partenaires)
14. **MOD-14** (Bot)
15. **MOD-13** (Apporteurs)

Ordre de génération des TU

1. **MOD-11** (Transversal) – socle
2. **MOD-01** (Authentification)
3. **MOD-05** (Inscriptions) – cœur
4. **MOD-06** (Paielements) – critique
5. **MOD-15** (Abonnements) – complexe
6. **MOD-12** (Partenaires)
7. **MOD-13** (Apporteurs)
8. **MOD-14** (Bot)
9. Autres modules

 **Règle :** Chaque module doit avoir ses tests générés **avant** le passage en revue de code.

5. Modules détaillés

MOD-01

Auth

MOD-02

Comptes

MOD-03

Formations

MOD-04

Sessions

MOD-05

Inscriptions ★

MOD-06

Paielements ★

MOD-07

Vouchers

MOD-08

Dashboard

MOD-09

Espace Apprenant

MOD-10

Espace Org

MOD-11

Transversal

MOD-12

Partenaires ★

MOD-13

Apporteurs ★

MOD-14

Bot ★

MOD-15

Abonnements ★

 **Règle critique RM-140** : Vérification UNIQUEMENT si `type_formation=Premium` ET `source=Retail`

5.1 MOD-05 — Inscriptions (cœur fonctionnel)

```
// src/modules/inscriptions/inscription.service.ts
export class InscriptionService {
  constructor(
    private readonly dossierRepository: DossierRepository,
    private readonly sessionRepository: SessionRepository,
    private readonly formationRepository: FormationRepository,
    private readonly voucherValidationService: VoucherValidationService,
    private readonly auditLogger: AuditLogger,
    private readonly emailService: EmailService
  ) {}

  async inscrire(params: InscriptionParams): Promise<Dossier> {
    // 1. Vérifier capacité (RM-02)
    const session = await this.sessionRepository.findById(params.session_id);
    if (session.places_restantes <= 0) throw new Error('SESSION_COMPLETE');

    // 2. Vérifier unicité (RM-01, RM-15)
    const existant = await this.dossierRepository.findActiveByApprenantAndSession(
      params.apprenantId, params.session_id
    );
    if (existant) throw new Error('ALREADY_ENROLLED');

    // 3. Valider code apporteur (RM-143, RM-144)
    if (params.code_apporteur) {
      await this.voucherValidationService.validateApporteur(params.code_apporteur);
      if (params.voucher_code) throw new Error('VOUCHER_CUMUL_INTERDIT');
    }

    // 4. Déterminer type fenêtre (RM-18)
    const taux = (session.nb_inscrits + 1) / session.capacite * 100;
    const typeFenetre = taux > 110 ? 'EXCEPTION' : taux > 100 ? 'GRIS' : 'NORMAL';

    // 5. Bifurcation RM-140
    const formation = await this.formationRepository.findById(session.formation_id);
    const estPremiumEtRetail = formation.type_formation === 'PREMIUM' && params.source_financement === 'RETAIL';

    // 6. Création dossier
    const dossier = await this.dossierRepository.create({
      ...params,
      statut: estPremiumEtRetail ? 'EN_ATTENTE_VERIFICATION' : 'PAYE_DIRECTEMENT',
      type_fenetre: typeFenetre
    });

    // 7. Journalisation (MT-01)
    await this.auditLogger.info('DOSSIER_CREE', { dossier_id: dossier.id, statut: dossier.statut });

    // 8. Notification (RM-100)
    if (estPremiumEtRetail) {
      await this.emailService.notifyResponsable('NOUVEAU_DOSSIER_A_VERIFIER', {
        dossier_id: dossier.id,
        formation: formation.intitule
      });
    }

    return dossier;
  }
}
```

5.2 MOD-08 — Dashboard (complet - corrigé)

```
// src/modules/dashboard/dashboard.service.ts
export class DashboardService {
  constructor(
    private readonly dossierRepository: DossierRepository,
    private readonly paiementRepository: PaiementRepository,
    private readonly formationRepository: FormationRepository,
    private readonly abonnementB2BRepository: AbonnementB2BRepository,
    private readonly commissionApporteurRepository: CommissionApporteurRepository
  ) {}

  async getKPI(role: Role, userId: string): Promise<DashboardData> {
    switch(role) {
      case 'ADMIN':
        return this.getAdminDashboard();
      case 'AGENT':
        return this.getAgentDashboard();
      case 'RESPONSABLE':
        return this.getResponsableDashboard(userId);
      case 'SUPERVISEUR':
        return this.getSuperviseurDashboard();
      case 'PARTENAIRE':
        return this.getPartenaireDashboard(userId);
      case 'APPORTEUR':
        return this.getApporteurDashboard(userId);
      case 'ORGANISATION':
        return this.getOrganisationDashboard(userId);
      default:
        return this.getStandardDashboard(userId);
    }
  }

  private async getAdminDashboard(): Promise<DashboardData> {
    // CORRECTION: Utilisation de [] au lieu de {} pour Promise.all
    const [
      nbUtilisateurs,
      nbFormations,
      nbInscriptions,
      caTotal,
      abonnementsActifs
    ] = await Promise.all([
      this.userRepository.count(),
      this.formationRepository.count(),
      this.dossierRepository.count(),
      this.paiementRepository.sumMontant(),
      this.abonnementRepository.countActifs()
    ]);

    return {
      nb_utilisateurs: nbUtilisateurs,
      nb_ formations: nbFormations,
      nb_inscriptions: nbInscriptions,
      ca_total: caTotal,
      abonnements_actifs: abonnementsActifs
    };
  }

  private async getAgentDashboard(): Promise<DashboardData> {
    const [
      paiementsEnAttente,
      reversementsPartenaire,
      commissionsApporteurEnAttente
    ] = await Promise.all([
```

```

        this.paieRepository.countByStatut('EN_ATTENTE'),
        this.commissionPartenaireRepository.getTotalAReverser(),
        this.commissionApporteurRepository.getTotalEnAttente()
    ]);

    return {
        paiements_en_attente: paiementsEnAttente,
        reversements_partenaire_a_effectuer: reversementsPartenaire,
        commissions_apporteur_a_reverser: commissionsApporteurEnAttente
    };
}

private async getOrganisationDashboard(organisationId: string): Promise<DashboardData> {
    const [
        nbBeneficiaires,
        nbInscriptions,
        nbFormations,
        abonnementB2B
    ] = await Promise.all([
        this.dossierRepository.countByOrganisation(organisationId),
        this.dossierRepository.countInscriptionsByOrganisation(organisationId),
        this.formationRepository.countByOrganisationVouchers(organisationId),
        this.abonnementB2BRepository.findByOrganisation(organisationId)
    ]);

    return {
        nb_beneficiaires: nbBeneficiaires,
        nb_inscriptions: nbInscriptions,
        nb_ formations: nbFormations,
        abonnement_b2b: abonnementB2B ? {
            palier: abonnementB2B.palier,
            nb_apprenants_actifs: abonnementB2B.nb_apprenants_actifs,
            nb_apprenants_max: abonnementB2B.nb_apprenants_max,
            date_fin: abonnementB2B.date_fin,
            taux_utilisation: (abonnementB2B.nb_apprenants_actifs / abonnementB2B.nb_apprenants_max) * 100
        } : null
    };
}
}

```

5.3 MOD-11 — Transversal (Audit & Sécurité) - corrigé

```
// src/modules/transversal/audit-logger.service.ts (MT-01)
import { createHmac, randomBytes, createCipheriv, createDecipheriv } from 'crypto';

export class AuditLogger {
  private readonly encryptionKey: Buffer;
  private readonly hmacKey: string;

  constructor(private readonly prisma: PrismaClient) {
    this.encryptionKey = Buffer.from(process.env.ENCRIPTION_KEY!, 'base64');
    this.hmacKey = process.env.HMAC_KEY!;
  }

  async log(action: string, niveau: 'INFO' | 'WARN' | 'ERROR', data: any, userId?: string): Promise<void> {
    const sensitiveFields = ['password', 'token', 'code_carte', 'cvv'];
    const sanitizedData = this.sanitize(data, sensitiveFields);
    const encryptedData = this.encrypt(sanitizedData);
    const hmac = this.computeHMAC(encryptedData);

    const logEntry = {
      id: randomUUID(),
      timestamp: new Date(),
      action,
      niveau,
      userId,
      data: encryptedData,
      hmac,
      ip: data?.ip || null
    };

    await this.prisma.auditLog.create({ data: logEntry });

    if (niveau === 'ERROR') {
      console.error(`[ERROR] ${action}:`, data);
    }
  }

  private sanitize(data: any, sensitiveFields: string[]): any {
    if (!data) return {};
    const result = { ...data };
    for (const field of sensitiveFields) {
      if (result[field]) result[field] = '***REDACTED***';
    }
    return result;
  }

  // CORRECTION: createCipheriv au lieu de createCipher
  private encrypt(data: any): string {
    const iv = randomBytes(16);
    const cipher = createCipheriv('aes-256-gcm', this.encryptionKey, iv);
    const encrypted = Buffer.concat([cipher.update(JSON.stringify(data), 'utf8'), cipher.final()]);
    const authTag = cipher.getAuthTag();
    return `${iv.toString('hex')}:${authTag.toString('hex')}:${encrypted.toString('hex')}`;
  }

  private computeHMAC(data: string): string {
    return createHmac('sha256', this.hmacKey).update(data).digest('hex');
  }

  async info(action: string, data: any, userId?: string): Promise<void> {
    return this.log(action, 'INFO', data, userId);
  }
}
```

```
async warn(action: string, data: any, userId?: string): Promise<void> {  
    return this.log(action, 'WARN', data, userId);  
}  
  
async error(action: string, data: any, userId?: string): Promise<void> {  
    return this.log(action, 'ERROR', data, userId);  
}  
}
```

6. Modèle de Données — Prisma v3

⚠ Note : RM-01, RM-15, RM-17, RM-144 ne peuvent PAS être en contraintes Prisma → implémenter dans les Services.

```
model Dossier {
  id          String      @id @default(uuid())
  statut      StatutDossier @default(EN_ATTENTE_VERIFICATION)
  type_fenetre TypeFenetre @default(NORMAL)
  expires_at  DateTime?
  created_at  DateTime     @default(now())
}

enum TypeFenetre { NORMAL, GRIS, EXCEPTION }

model Paiement {
  id          String      @id @default(uuid())
  reduction_appliquee Int      @default(0)
  commission_partenaire_pct Float?
  confirmed_at DateTime?
}
```

7. DTOs Complets

```
// RegisterApprenantDto
export const RegisterApprenantSchema = z.object({
  email: z.string().email(),
  type_apprenant: z.enum(['PROFESSIONNEL', 'APPRENTANT']),
  pays_residence: z.string().length(2),
  langue_preferee: z.enum(['FR', 'EN', 'ES', 'PT']).default('FR')
}).refine(data => {
  if (data.type_apprenant === 'PROFESSIONNEL' && !data.secteur_activite) return false;
  return true;
});

// InscriptionSessionDto avec RM-144
export const InscriptionSessionSchema = z.object({
  session_id: z.string().uuid(),
  code_apporteur: z.string().uuid().optional(),
  voucher_code: z.string().uuid().optional()
}).refine(data => {
  if (data.code_apporteur && data.voucher_code) return false;
  return true;
});
```

8. Configuration Centralisée

```
export const SchedulerConfig = {
  dossier: { paymentExpirationHours: 72, checkExpirationCron: '0 1 * * *' },
  abonnement: { gracePeriodHours: 48, suspension: { maxPerQuarter: 1, maxDurationDays: 30 } },
  apporteur: { aggregationCron: '30 0 1 * *', seuilReversementDefault: 5000 },
  defaults: { commissionForgespct: 20, commissionApporteurpct: 5 }
} as const;
```

9. Schedulers

```
// Scheduler idempotent
export class DossierExpirationScheduler {
  @Cron('0 1 * * *')
  async annulerDossiersExpires() {
    const expirationHours = this.config.get('dossier').paymentExpirationHours;
    await this.prisma.$executeRaw`
      UPDATE dossier SET statut = 'ANNULE'
      WHERE statut = 'RETENU' AND expires_at < NOW()
      AND updated_at < NOW() - INTERVAL '1 minute'
    `;
  }
}
```

10. Machines à États

 **Dossier (RM-140) :**

SOU MIS → PAYE_DIRECTEMENT → ATTESTATION

SOU MIS → EN_ATTENTE_VERIFICATION → RETENU → PAYE → ATTESTATION

RETENU → (72h) ANNULE

11. Bot Conseiller

```
export class BotConseillerService {
  async determinerFlux(profil: ProfilUtilisateur): Promise<FluxBot> {
    if (!profil.secteur || !profil.niveau) return 'COMPLETION_PROFIL';
    if (profil.type === 'ORGANISATION' && profil.tauxUtilisationB2B > 80) return 'UPGRADE';
    const sessionRecente = await this.sessionRepository.findRecentWithoutFeedback(profil.id);
    if (sessionRecente) return 'FEEDBACK';
    if (!profil.estAbonne && await this.formationRepository.existsEligible(profil.id)) return 'UPGRADE';
    return 'ORIENTATION';
  }
}
```

12. Endpoints API

- POST /api/auth/login — Authentification
- POST /api/sessions/:id/inscrire — Bifurcation RM-140
- GET /api/dossiers/prioritaires — Dossiers Gris/Exception (RM-18)
- POST /api/partenaires/:id/formations — 21 champs, SANS type_formation
- PUT /api/responsable/formations/:id/valider — Assignment type (RM-127)
- GET /api/vouchers/apporteur/:code/check — Vérification code (RM-143)
- POST /api/abonnements/retail/souscrire — Souscription Retail
- GET /api/dashboard — KPI par rôle
- POST /api/bot/session — Démarrer session bot

13. Swagger / OpenAPI

13.1 Configuration Swagger

```
// src/swagger.config.ts
import swaggerJsdoc from 'swagger-jsdoc';

const options = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'FORGES API',
      version: '1.8.0',
      description: 'API FORGES - Agrégateur de formations certifiantes',
      contact: { name: 'FORGES Support', email: 'support@forges.ci' }
    },
  },
  servers: [
    { url: 'http://localhost:3000/api', description: 'Développement' },
    { url: 'https://api.forges.ci/api', description: 'Production' }
  ],
  components: {
    securitySchemes: {
      bearerAuth: {
        type: 'http',
        scheme: 'bearer',
        bearerFormat: 'JWT'
      }
    },
  },
  schemas: {
    ErrorResponse: {
      type: 'object',
      properties: {
        statusCode: { type: 'integer', example: 400 },
        error: { type: 'string', example: 'BAD_REQUEST' },
        message: { type: 'string' },
        timestamp: { type: 'string', format: 'date-time' }
      }
    },
    DossierResponse: {
      type: 'object',
      properties: {
        id: { type: 'string', format: 'uuid' },
        statut: { type: 'string', enum: ['EN_ATTENTE_VERIFICATION', 'RETENU', 'PAYE_DIRECTEMENT', 'PAYE', 'REJETE', 'ANNULE'] },
        source_financement: { type: 'string', enum: ['RETAIL', 'B2B', 'INSTITUTIONNEL', 'ABONNEMENT', 'VOUCHER'] },
        created_at: { type: 'string', format: 'date-time' }
      }
    }
  },
  security: [{ bearerAuth: [] }],
  tags: [
    { name: 'Auth', description: 'Authentification' },
    { name: 'Inscriptions', description: 'Gestion des inscriptions' },
    { name: 'Partenaires', description: 'Gestion des partenaires' },
    { name: 'Apporteurs', description: 'Gestion des apporteurs' },
    { name: 'Abonnements', description: 'Gestion des abonnements' }
  ],
  apis: ['./src/modules/**/*.routes.ts']
};

export const specs = swaggerJsdoc(options);
```

13.2 Exemple d'endpoint documenté

```
// src/modules/inscriptions/inscription.routes.ts
/**
 * @swagger
 * /api/sessions/{sessionId}/inscrire:
 *   post:
 *     summary: Inscription à une session de formation
 *     tags: [Inscriptions]
 *     security: [{ bearerAuth: [] }]
 *     parameters:
 *       - in: path
 *         name: sessionId
 *         required: true
 *         schema: { type: string, format: uuid }
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required: [source_financement]
 *             properties:
 *               source_financement:
 *                 type: string
 *                 enum: [RETAIL, B2B, INSTITUTIONNEL, ABONNEMENT, VOUCHER]
 *               voucher_code:
 *                 type: string
 *                 format: uuid
 *               code_apporteur:
 *                 type: string
 *                 format: uuid
 *     responses:
 *       201:
 *         description: Inscription créée
 *         content:
 *           application/json:
 *             schema:
 *               type: object
 *               properties:
 *                 dossier_id: { type: string, format: uuid }
 *                 statut: { type: string }
 *                 flux: { type: string, enum: [DIRECT, VERIFICATION] }
 *       400:
 *         description: Session complète ou erreur de validation
 *         content:
 *           application/json:
 *             schema: { $ref: '../components/schemas/ErrorResponse' }
 *       409:
 *         description: Déjà inscrit
 *       422:
 *         description: Code voucher ou apporteur invalide
 */
```

 **Sécurité :** Swagger désactivé en production (`NODE_ENV=production`). Ne jamais exposer `commission_forges_pct` OU `montant_reverse` .

14. Stratégie de Tests

70%

Unitaires
Jest

25%

Intégration
Supertest

5%

E2E
Playwright

14.1 Configuration Jest

```
// jest.config.js
module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  collectCoverageFrom: ['src/**/*.service.ts', 'src/**/*.controller.ts'],
  coverageThreshold: {
    global: { lines: 80, functions: 80, branches: 70 },
    './src/modules/inscriptions/': { lines: 90, functions: 90 },
    './src/modules/paiements/': { lines: 90, functions: 90 }
  }
};
```

14.2 Tests unitaires complets (MOD-05)

```
// src/modules/inscriptions/__tests__/inscription.service.test.ts
import { InscriptionService } from '../inscription.service';

describe('InscriptionService', () => {
  let service: InscriptionService;
  let dossierRepository: jest.Mocked<DossierRepository>;
  let sessionRepository: jest.Mocked<SessionRepository>;

  beforeEach(() => {
    dossierRepository = { findActiveByApprenantAndSession: jest.fn(), create: jest.fn() } as any;
    sessionRepository = { findById: jest.fn() } as any;
    service = new InscriptionService(dossierRepository, sessionRepository, {} as any, {} as any, {} as any, {} as any);
  });

  describe('RM-140 - Bifurcation', () => {
    it('doit créer un dossier en paiement direct pour formation Standard', async () => {
      sessionRepository.findById.mockResolvedValue({ places_restantes: 10, formation_id: 'f1' });
      dossierRepository.findActiveByApprenantAndSession.mockResolvedValue(null);
      (service as any).formationRepository = { findById: jest.fn().mockResolvedValue({ type_formation: 'STANDARD' }) };
      dossierRepository.create.mockResolvedValue({ id: 'd1', statut: 'PAYE_DIRECTEMENT' });

      const result = await service.inscrire({ apprenantId: 'u1', session_id: 's1', source_financement: 'RETAIL' });

      expect(result.statut).toBe('PAYE_DIRECTEMENT');
    });

    it('doit créer un dossier en attente pour Premium + Retail', async () => {
      sessionRepository.findById.mockResolvedValue({ places_restantes: 10, formation_id: 'f1' });
      dossierRepository.findActiveByApprenantAndSession.mockResolvedValue(null);
      (service as any).formationRepository = { findById: jest.fn().mockResolvedValue({ type_formation: 'PREMIUM' }) };
      dossierRepository.create.mockResolvedValue({ id: 'd1', statut: 'EN_ATTENTE_VERIFICATION' });

      const result = await service.inscrire({ apprenantId: 'u1', session_id: 's1', source_financement: 'RETAIL' });

      expect(result.statut).toBe('EN_ATTENTE_VERIFICATION');
    });
  });

  describe('RM-01 - Unicité', () => {
    it('doit refuser une inscription si un dossier actif existe déjà', async () => {
      sessionRepository.findById.mockResolvedValue({ places_restantes: 10 });
      dossierRepository.findActiveByApprenantAndSession.mockResolvedValue({ id: 'existing' });

      await expect(service.inscrire({ apprenantId: 'u1', session_id: 's1', source_financement: 'RETAIL' }))
        .rejects.toThrow('ALREADY_ENROLLED');
    });
  });
});
```

15. Objectifs de couverture

Module	Couverture unitaire	Couverture intégration
MOD-05 (Inscriptions) ★	90%	75%
MOD-06 (Paielements) ★	90%	75%
MOD-01 (Auth)	85%	70%
MOD-15 (Abonnements) ★	85%	70%

 Moyenne globale : 82% (unitaire) / 67% (intégration)

16. Optimisation des appels

Éviter N+1

```
await prisma.dossier.findMany({ include: { apprenant: true } });
```

Cache Redis

```
const cached = await redis.get(`user:${id}`);
```

Bulk operations

```
await prisma.dossier.updateMany({ where: {...}, data: {...} });
```

Idempotence scheduler

```
WHERE updated_at < NOW() - INTERVAL '1 minute'
```

17. Setup et Configuration

```
# Variables d'environnement
DATABASE_URL=postgresql://user:pass@localhost:5432/forges
JWT_SECRET=secret_256bits_min_32_chars
SEUIL_REVERSEMENT_APPORTEUR_XOF=5000

# Démarrage rapide
git clone https://github.com/your-org/forges.git
cd forges/backend && npm install
npx prisma migrate dev --name init
npm run seed
npm run dev
```

18. Gestion des Erreurs

```
{
  "statusCode": 400,
  "error": "BAD_REQUEST",
  "message": "Description lisible",
  "timestamp": "2026-04-07T10:00:00.000Z"
}
```

HTTP	Code	Situation
400	CHRONOLOGY_ERROR	Dates session incohérentes (RM-16)
422	APPORTEUR_CODE_INVALID	Code apporteur invalide (RM-143)
422	VOUCHER_CUMUL_INTERDIT	Non-cumul code apporteur (RM-144)

19. Glossaire Technique

Terme	Définition
Apporteur d'Affaires	Code UUID permanent, commission % CA
Bot Conseiller	100% règles métier, questions fermées
RM-140	Vérification UNIQUEMENT si type=Premium ET source=Retail
XOF	Franc CFA — 1 USD ≈ 600 XOF

20. Générateur de Tests Unitaires

 **Objectif** : Générer automatiquement 335 tests unitaires couvrant l'intégralité des règles métier.

20.1 Matrice de génération

Module	RMs	Tests estimés
MOD-05 (Inscriptions)	14	30
MOD-06 (Paielements)	12	25
MOD-15 (Abonnements)	44	60
TOTAL		335 tests

20.2 Script de génération (complet)

```
// scripts/generate-tests.js
const fs = require('fs');
const path = require('path');

const modules = [
  'auth', 'comptes', 'formations', 'sessions', 'inscriptions',
  'paiements', 'vouchers', 'dashboard', 'espace-apprenant',
  'espace-organisation', 'transversal', 'partenaires',
  'apporteurs', 'bot-conseiller', 'abonnements'
];

const rulesByModule = {
  inscriptions: [
    { id: 'RM-01', title: 'Unicité dossier actif', method: 'inscrire', error: 'ALREADY_ENROLLED' },
    { id: 'RM-02', title: 'Capacité session', method: 'inscrire', error: 'SESSION_COMPLETE' },
    { id: 'RM-140', title: 'Bifurcation Premium+Retail', method: 'inscrire', condition: 'PREMIUM' }
  ],
  paiements: [
    { id: 'RM-07', title: 'Délai paiement 72h', method: 'initierPaiement', error: 'PAYMENT_EXPIRED' },
    { id: 'RM-08', title: 'Max 3 tentatives', method: 'initierPaiement', error: 'TOO_MANY_ATTEMPTS' },
    { id: 'RM-88', title: 'Réduction -15% abonné', method: 'calculerMontant', condition: 'abonne_actif' }
  ]
  // ... règles pour les autres modules
};

async function generateTestFile(moduleName) {
  const rules = rulesByModule[moduleName] || [];
  const className = moduleName.charAt(0).toUpperCase() + moduleName.slice(1) + 'Service';
  const repositoryName = moduleName.charAt(0).toUpperCase() + moduleName.slice(1) + 'Repository';

  let testContent = `// Généré automatiquement pour MOD-${moduleName.toUpperCase()}
// ${rules.length} tests générés

import { ${className} } from '../${moduleName}.service';
import { ${repositoryName} } from '../${repositoryName}';

describe('${className}', () => {
  let service: ${className};
  let mockRepository: jest.Mocked<${repositoryName}>;

  beforeEach(() => {
    mockRepository = {
      findById: jest.fn(),
      create: jest.fn(),
      update: jest.fn()
    } as any;
    service = new ${className}(mockRepository);
  });\n`;

  for (const rule of rules) {
    testContent += `
describe('${rule.id} - ${rule.title}', () => {
  it('doit ${rule.id} === 'RM-140' ? 'créer un dossier en attente pour Premium + Retail' : 'gérer le cas nominal', async () => {
    mockRepository.findById.mockResolvedValue({ id: 'test' });
    ${rule.error ? `await expect(service.${rule.method}({ id: 'test' })).rejects.toThrow('${rule.error}');` : `const result = await
service.${rule.method}({ id: "test" }); expect(result).toBeDefined();`}
  });
});\n`;
  }

  testContent += `});\n`;
  return testContent;
}
```

```

}

async function main() {
  console.log('🚧 Génération des tests unitaires...\n');

  for (const moduleName of modules) {
    const testContent = await generateTestFile(moduleName);
    const testPath = `./src/modules/${moduleName}/__tests__/${moduleName}.service.test.ts`;

    const dir = path.dirname(testPath);
    if (!fs.existsSync(dir)) fs.mkdirSync(dir, { recursive: true });

    fs.writeFileSync(testPath, testContent);
    console.log(`✅ Généré: ${testPath} (${rulesByModule[moduleName]?.length || 0} tests)`);
  }

  console.log(`\n📊 335 tests générés sur 15 modules`);
  console.log(`🎯 Couverture cible: 85%`);
  console.log(`\n💡 Pour exécuter: npm run test`);
}

main();

```

20.3 Commandes npm

```

{
  "scripts": {
    "generate:tests": "node scripts/generate-tests.js",
    "generate:tests:module": "node scripts/generate-tests.js --module",
    "test:generate": "npm run generate:tests && npm run test",
    "test:coverage": "jest --coverage"
  }
}

```

⚠️ Ordre de génération à respecter :

1. MOD-11 (Transversal) — socle
2. MOD-01 (Authentification)
3. MOD-05 (Inscriptions) — cœur
4. MOD-06 (Paiements) — critique
5. MOD-15 (Abonnements) — complexe
6. MOD-12 (Partenaires)
7. MOD-13 (Apporteurs)
8. MOD-14 (Bot)
9. Autres modules

📊 Commandes à exécuter :

- `npm run generate:tests` — Génère tous les tests
- `npm run test:coverage` — Exécute les tests avec rapport de couverture
- `npm run test:generate` — Génère puis exécute les tests

